

Mathematical notation

Last modified: 2026-10-08 21:07:14 (PDT)

AI-Assisted Authorship

This chapter was written with the assistance of GitHub Copilot, which expanded on outlined ideas and draft notes provided by the author. The content represents the author's perspective and has been reviewed for accuracy, but the detailed prose was generated through AI assistance.

Notation is part of your prose, and readers parse it the same way they parse a sentence. This chapter collects notational habits that make quantitative writing easier to read.

Avoiding double inequalities

A *double inequality* (or *chained inequality*) places one quantity between two others, as in $a < x < b$. Prefer interval notation instead: $x \in (a, b)$.

Interval notation is easier to read for three reasons:

- It states one relation instead of two, so the reader parses a single claim rather than combining a pair of them.
- It names the set you actually mean, so the same expression can be reused as a domain, a constraint, or the argument of a quantifier.
- It marks inclusive and exclusive endpoints with brackets and parentheses, which are easier to distinguish at a glance than $\leq$ and $<$.

Example 0.1 (Rewriting a double inequality).

Suppose $0 \leq p \leq 1$ and $0 < \sigma < \infty$.

Suppose $p \in [0, 1]$ and $\sigma \in (0, \infty)$.

The rewrite makes the two constraints look alike, so the reader can compare them directly instead of checking four inequality symbols.

The same guidance applies to prose: write “for $x \in (0, 1)$ ” rather than “for x between 0 and 1, exclusive”.

When the middle term is not a single symbol

Interval notation works best when the quantity being constrained is a single expression. When the middle term is complicated, naming it first keeps the constraint short.

Example 0.2 (Naming the middle term).

$$0 < \frac{\hat{\theta} - \theta}{\text{se}(\hat{\theta})} < 1.96$$

Let $z = \frac{\hat{\theta} - \theta}{\text{se}(\hat{\theta})}$; then $z \in (0, 1.96)$.

Finally, a chain that compares three *different* quantities — for example $\hat{\theta}_1 < \hat{\theta}_2 < \hat{\theta}_3$ — is not a constraint on a single quantity, so interval notation does not apply. Write such comparisons as separate statements ($\hat{\theta}_1 < \hat{\theta}_2$ and $\hat{\theta}_2 < \hat{\theta}_3$) unless the ordering of all three is the point you are making.

One operation per step

Write a derivation so that each displayed line does exactly one thing to the line before it: apply one rule, substitute one result, or simplify one piece. Give each line its own justification.

Combining steps saves the writer a line and costs every reader a calculation. A reader who cannot see how one line follows from the last has to redo the missing work on scratch paper, or take the step on trust. Either way, an error hidden inside a combined step is harder for the reader, or a reviewer, to catch. The advice on “[Trivial](#)” and “[trivially](#)” says to write out a step that takes only a line; this section applies that advice to the layout of a displayed derivation.

Example 0.3 (Splitting a combined step). Suppose a derivation has reached $-2 \sum_{i=1}^n (y_i - \beta_0 - \beta_x x_i)$.

$$-2 \sum_{i=1}^n (y_i - \beta_0 - \beta_x x_i) = -2(n\bar{y} - n\beta_0 - \beta_x n\bar{x})$$

$$\begin{aligned}
-2 \sum_{i=1}^n (y_i - \beta_0 - \beta_x x_i) &= -2 \left(\sum_{i=1}^n y_i - \sum_{i=1}^n \beta_0 - \sum_{i=1}^n \beta_x x_i \right) && \text{(split the sum)} \\
&= -2 \left(\sum_{i=1}^n y_i - n\beta_0 - \sum_{i=1}^n \beta_x x_i \right) && \text{(sum of a constant)} \\
&= -2 \left(\sum_{i=1}^n y_i - n\beta_0 - \beta_x \sum_{i=1}^n x_i \right) && \text{(constant factor out of the sum)} \\
&= -2 \left(n\bar{y} - n\beta_0 - \beta_x \sum_{i=1}^n x_i \right) && \left(\sum_i y_i = n\bar{y} \right) \\
&= -2(n\bar{y} - n\beta_0 - \beta_x n\bar{x}) && \left(\sum_i x_i = n\bar{x} \right)
\end{aligned}$$

The `one_line` version does five operations in one line: it splits the sum, sums a constant, factors a constant out of a sum, substitutes $n\bar{y}$, and substitutes $n\bar{x}$. The `five_lines` version gives each of the five its own line.

Common ways to combine steps without noticing:

- applying the rule for the derivative of a sum and the chain rule in the same line;
- substituting a result and then simplifying it;
- multiplying constants and factoring them out of a sum;
- expanding a product and collecting like terms;
- canceling a in $a + (b - a)$ removes the parentheses $(a + b - a)$, reorders the terms $(b + a - a)$, groups the two that cancel $(b + (a - a))$, replaces $a - a$ with 0 $(b + 0)$, and drops the $+0$;
- canceling n in $n \cdot \frac{b}{n}$ rewrites the division as multiplication by a reciprocal $(n \cdot (b \cdot \frac{1}{n}))$, removes the parentheses $(n \cdot b \cdot \frac{1}{n})$, reorders the factors $(b \cdot n \cdot \frac{1}{n})$, groups the two that cancel $(b \cdot (n \cdot \frac{1}{n}))$, replaces $n \cdot \frac{1}{n}$ with 1 $(b \cdot 1)$, and drops the $\cdot 1$;
- writing “setting this to zero and dividing by $-2n$ gives”, which hides two operations in a sentence.

Using a fact the reader has not seen

When a step relies on a fact the document has not yet shown, such as deviations from a mean summing to zero, prove that fact first, in its own short derivation or exercise, and cite that proof at the step that uses the fact. Don’t leave the fact as an unexplained justification.

Long derivations

Splitting every step makes derivations longer. Keep them readable by breaking them into stages: derive an intermediate result in its own block, then plug that result back into the main derivation. For example, when you apply the chain rule, derive the inner derivative in a separate block before you substitute it. Naming a part of an expression that recurs in every line, such as writing d_i for $(y_i - \bar{y}) - \beta_x(x_i - \bar{x})$, can also keep each line short.

Functions and their values

A function and its value at a point are different objects. In $\mu_i = \mu(x_i)$, μ is a function, and $\mu(x_i)$ is a number: the value of μ at x_i . Written with its placeholder argument, $\mu(x)$ already denotes a value, so “the value of $\mu(x)$ at x_i ” reads as a value of a value. Either drop the placeholder and write “the value of μ at x_i ”, or keep the placeholder and name the substitution: “ $\mu(x)$ evaluated at x_i ”.

Example 0.4 (Saying where a function is evaluated).

Each outcome is centered on the value of a mean function $\mu(x)$ at its own covariate values.

Each outcome is centered on a mean function $\mu(x)$ evaluated at that outcome’s covariate values.

the value of the density $f(x)$ at 0

the value of f at 0, or $f(0)$

the likelihood $L(\theta)$ at $\hat{\theta}$

$L(\theta)$ evaluated at $\hat{\theta}$, or $L(\hat{\theta})$

In each version, “the value of” or “at” sits next to a placeholder argument, so the reader has to decide whether $\mu(x)$, $f(x)$ or $L(\theta)$ means the function or one of its values. The first version also has an ambiguous “its”: the nearest candidate is the mean function, not the outcome.

Use the notation to match:

- Write μ , or “the mean function”, for the function itself.
- Write $\mu(x)$ with a placeholder argument where you introduce or define the function, to show what its argument is, as in “a mean function $\mu(x)$ ” or $\mu(x) := \beta_0 + \beta_x x$, and say “evaluated at” when you substitute a point into it.
- Write $\mu(x_i)$, or “the value of μ at x_i ”, for a value.

Matching names and notation

A name and its notation must describe the same object. When you name a quantity in words and also show it in symbols, use the symbols that go with that name. A reader who sees the name “dot product” expects to see the notation for a dot product. If the symbols look like a different operation, the reader has to work out whether the name or the symbols is wrong.

Keep the pairing the same everywhere the quantity appears. Notes, assignments, solutions and slides in one course should use one name and one notation for each quantity. When two fields call one idea by different names, choose one name, list the other names where you define the idea, and use only the chosen name afterward (see [Name the synonyms](#)).

Example 0.5 (Matching a name to its notation).

Compute the dot product $\tilde{a}'\tilde{b}$.

Compute the dot product $\tilde{a} \cdot \tilde{b}$.

The first version names a dot product but writes a matrix product with a transpose. The two are equal for column vectors, but the reader sees two names for one quantity. If you want to show that they are equal, say so in a sentence, as in “the dot product $\tilde{a} \cdot \tilde{b}$ equals the matrix product $\tilde{a}'\tilde{b}$ ”. [Writing dot products](#) says when to use each form.

In a course that writes regression in statistical notation:

Consider the model $h(x) = wx + b$.

Consider the mean function $\mu(x) = \beta_0 + \beta_x x$.

The second version names the same quantity as the rest of the course with a different word and different symbols. A student then has to learn that h , w and b are μ , β_x and β_0 .

Before you release a document that reuses a quantity from another one, search both for its name and its symbols, and make them agree.

Using shared macros

Write each symbol through a shared macro file, and write the macro everywhere the symbol appears. A group that writes the same quantity in raw LaTeX in one document and with a macro in another ends up with two notations for one quantity. Changing a convention then means editing every document by hand.

Before you write any symbol, search the group’s shared macro file for the concept it names. If a macro names the concept, use it. If none does, add one to the shared file before you use the symbol, so the next document finds it.

Example 0.6 (Using a shared macro). This example compares the source you type, so both versions appear as code.

Source: Let `\beta_1` be the slope and write the sum as `\sum_{i=1}^n`.

Source: Let `\coef{1}` be the slope and write the sum as `\sumin`.

The version writes the slope and the sum in raw LaTeX. The version names each concept once, in the shared file. When the group changes how it prints a coefficient or a sum, every document that uses the macro changes with it.

The same rule covers these quantities:

- transposes
- hats on estimates
- expectations
- variances
- probabilities
- vectors

A quantity that has a macro keeps the same symbol in every document.

Name a macro for what the symbol means, not for the letter it prints. A macro such as `\vdelta` or `\hbeta` only spells a Greek letter, so it names no concept, and changing the symbol for that concept still means editing every use. Outside a passage that discusses the notation itself, do not write a Greek letter in a math expression, either as a raw command such as `\delta` or inside a letter-named macro such as `\vdelta`.

Example 0.7 (Naming a macro for its meaning).

Source: The error at layer `\ell` is `\vdelta^{(\ell)}`, and the step size is `\eta`.

Source: The error at layer `\ell` is `\backerr^{(\ell)}`, and the step size is `\learnrate`.

Both versions print the same symbols. In the version, a reader of the source sees what each symbol stands for, and a group that changes the symbol for the step size changes one line in the shared file.

A passage about the notation itself, such as a table that lists each symbol and what it means, may write the letters directly.

The same reasoning applies to decorators. A hat, a tilde or a bar says how a symbol is drawn, not what it means. A hat can mark an estimate, a fitted value or a Fourier transform, so `\hat` names no concept either. Write the macro that names the role instead, such as `\est{}` for an estimate, and keep `\hat` for passages that discuss the notation itself. New math follows this now; existing hats are converted in separate sweeps.

Example 0.8 (Naming a decorator for its meaning).

Source: The estimate $\hat{\text{mean}}$ is close to mean .

Source: The estimate $\text{est}\{\text{mean}\}$ is close to mean .

Both versions print the same hat. If the group later marks estimates some other way, only the definition of `\est` changes.

Latin letters with a fixed role follow the same rule. In a regression or prediction model, use these macros:

- `\outvar` for the outcome variable, instead of a bare `y`;
- `\eoutvar` for a predicted outcome;
- `\predvar` for a predictor, instead of a bare `x`;
- `\resid` for a residual, the observed minus the predicted outcome;
- `\prederr` for a prediction error, the predicted minus the observed outcome.

For an observation used to fit the model, the residual is the negative of the prediction error. Write `\resid` for the observed minus the predicted outcome and `\prederr` for the predicted minus the observed outcome, never one for the other. For an estimate of a parameter, `\erf{\eparam}` is the estimation error, the estimate minus the true value; it prints ε , the symbol the shared macros use for an error. These macros need a version of the shared macros that includes them, so update a repository's copy before using them there. The outcome, predictor and residual macros also have random-variable and vector forms, such as `\Outvar`, `\voutvar` and `\vresid`.

Example 0.9 (Naming a variable for its role).

Source: The residual is $e_i = y_i - \hat{y}_i$.

Source: The residual is $\text{resid}_i = \text{outvar}_i - \text{eoutvar}_i$.

The first version prints the residual as r , the symbol the shared macros use for it; the other letters are the same in both. In the second version, a reader of the source can tell which letter is the outcome and which is the residual.

Once a quantity has a name and a macro, write the macro instead of the expression that defines it. Each equation then stays short, and a reader sees which quantity it is about.

Example 0.10 (Writing a named quantity by its name).

Source: The gradient of the mean squared error is $\frac{2}{n} \text{tp}\{\text{design}\} \text{paren}\{\text{design}\text{vcoef} - \text{vy}\}$.

Source: The gradient of the mean squared error is $\frac{2}{n} \text{tp}\{\text{design}\} \text{vprederr}$.

The macro `\vprederr` prints the vector of prediction errors. The document defines it once, where the model is introduced, as `\vprederr = \design\vcoef - \vy`. The `version` has one fewer level of brackets, and it names the quantity the gradient depends on.

Writing dot products

When two vectors multiply to give a number, write the product as a dot product, $\tilde{x} \cdot \tilde{\beta}$. Do not write it as a transpose product, $\tilde{x}'\tilde{\beta}$, or with inner-product brackets, $\langle \tilde{x}, \tilde{\beta} \rangle$. The dot shows one operation on two vectors. A transpose product asks the reader to picture a row vector times a column vector and to work out that the result is a single number.

Example 0.11 (Writing a dot product).

The model is $f(\tilde{x}) = \tilde{x}'\tilde{\beta}$.

The model is $f(\tilde{x}) = \tilde{x} \cdot \tilde{\beta}$.

Both versions give the same number. The `version` shows it as one product of two vectors. In a group that uses shared macros (see [Using shared macros](#)), write the `version` with the dot-product macro, for example $\text{dprod}\{\text{vx}\}\{\text{vbeta}\}$.

Keep the transpose where the product is not a dot product of two vectors:

- a product of matrices, such as $X'X$
- an outer product, such as $\tilde{x}\tilde{x}'$
- a quadratic form, such as $\tilde{x}'A\tilde{x}$

Keep the inner-product brackets where the text means any inner product, such as a definition of an inner product space.

Writing out notational shorthands

A notational shorthand drops part of an expression that the writer expects the reader to fill in. The commonest case is leaving the limits off a sum, product, or integral, as in

$\sum_x f(x)$. In permanent writing (notes, papers, and published slides), write the full form instead: $\sum_{x \in \mathcal{R}(X)} f(x)$, where $\mathcal{R}(X)$ is the set of values that X can take.

The full form costs the writer a few characters and saves every reader a guess. A reader who sees $\sum_x$ has to work out which values of x the sum runs over, and readers who work out different answers will disagree about what the expression means. The full form also exposes mistakes. If a count variable takes only non-negative values, a sum written as $\sum_{x \in \mathbb{Z}}$ shows the wrong set at a glance, while $\sum_x$ hides the same mistake.

Give the limits of every sum, product, and integral, and state the set that any other index runs over. Board work during a lecture is often less complete, but it should still aim for the full form. The [notation page of the Math for Data Science notes](#) catalogs common shorthands and their full forms.

Example 0.12 (Writing out the limits of a sum and the range of an index). For a discrete random variable X :

$$\begin{aligned} \mathbb{E}[X] &= \sum_x x \Pr(X = x) \\ \mathbb{E}[X] &= \sum_{x \in \mathcal{R}(X)} x \Pr(X = x) \end{aligned}$$

For the likelihood of n independent observations $x_1, \dots, x_n$:

$$\begin{aligned} L(\theta) &= \prod_i f(x_i; \theta) \\ L(\theta) &= \prod_{i=1}^n f(x_i; \theta) \end{aligned}$$

Each full form tells the reader the set the operator runs over: the values X can take, and the indices of the n observations.